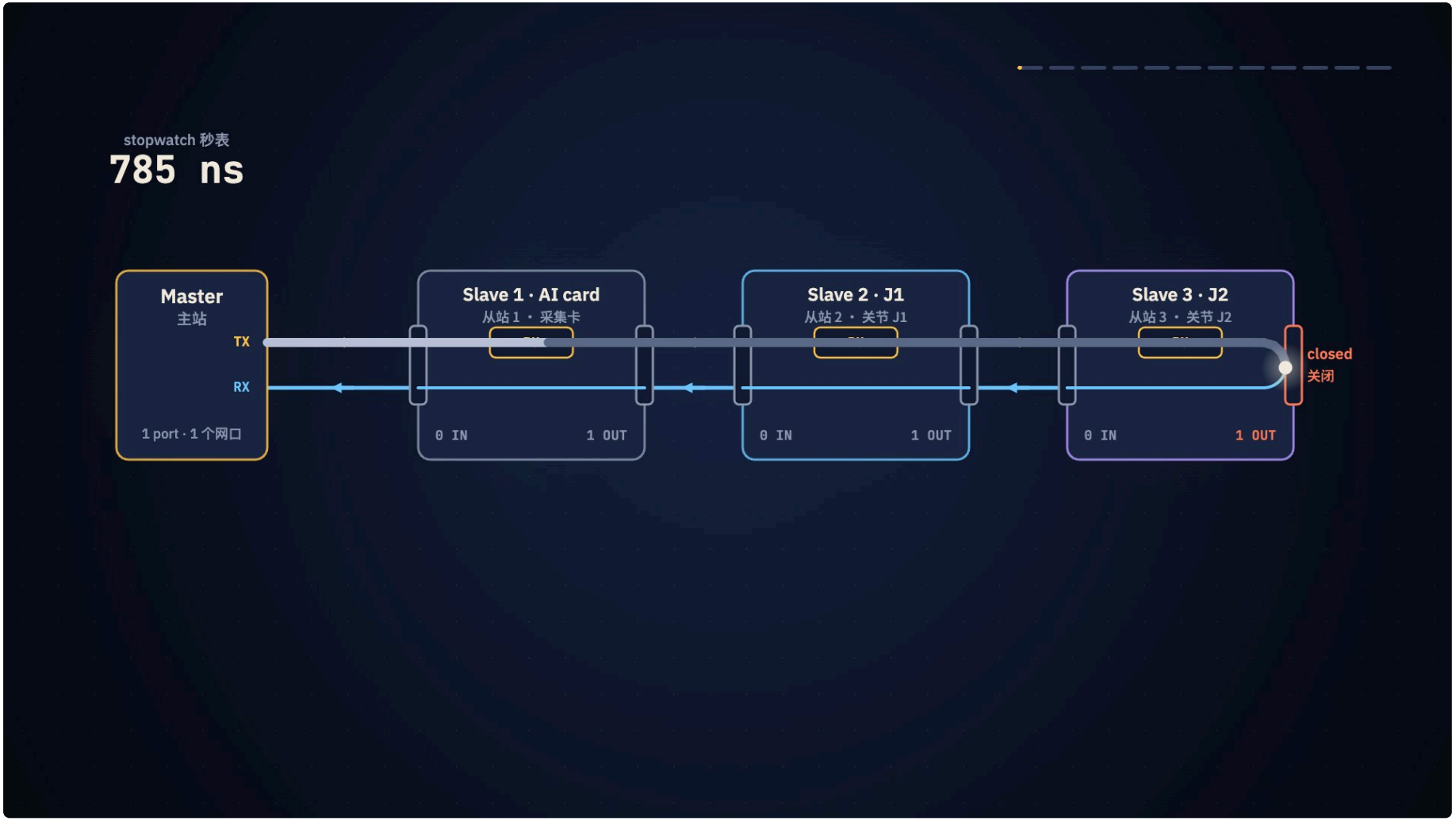


001 · 还没出门就到家了 · Home before it has left 0:00

上一集留了一个谜题。

Last episode ended on a puzzle.



002 · 还没出门就到家了 · Home before it has left 0:05

我们这一帧的第一个比特，1.5 微秒就回到了主站。

Our frame's first bit is back at the master after 1.5 µs.

可整帧发出去要 8 微秒。

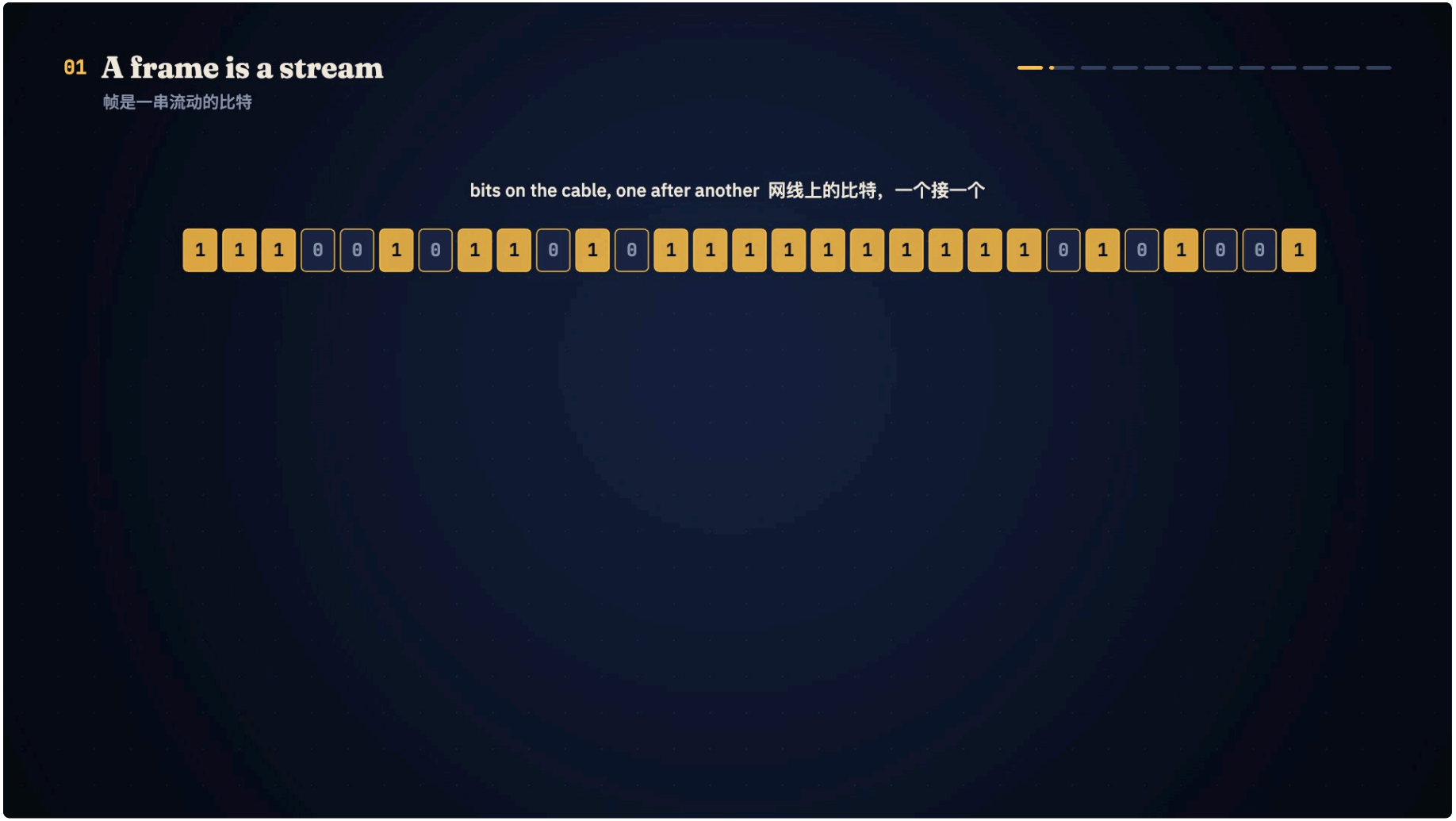
But sending the whole frame takes 8 µs.



003 · 还没出门就到家了 · Home before it has left 0:16

帧头怎么能在帧尾出门之前就到家?

How can the head be home before the tail has even left?



004 · 帧是一串流动的比特 · A frame is a stream 0:24

在网线上，帧不是一个包裹，而是一串接一串的比特。

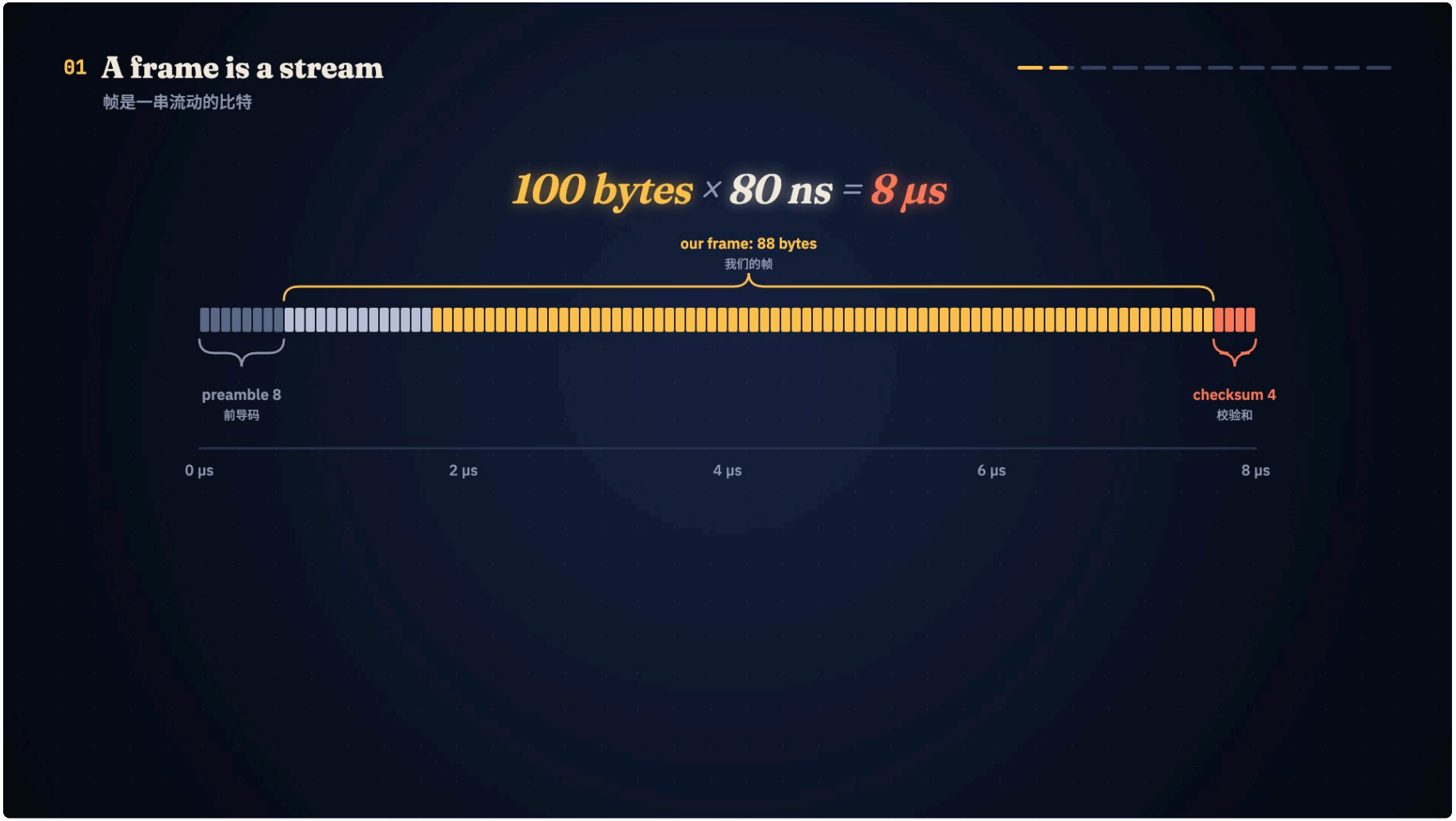
On the cable, a frame is not a parcel. It is a stream of bits, one after another.



005 · 帧是一串流动的比特 · A frame is a stream 0:33

100 Mbit/s 下，一个比特 10 纳秒，一个字节 80 纳秒。

At 100 Mbit/s, one bit takes 10 ns. One byte takes 80 ns.



006 · 帧是一串流动的比特 · A frame is a stream 0:40

我们的周期帧 88 字节，加上前导码和校验和，线上一共 100 字节。

Our cyclic frame is 88 bytes. With its preamble and checksum, 100 bytes go on the wire.

100 字节 × 80 纳秒 = 8 微秒，从第一个比特到最后一个。

100 bytes × 80 ns = 8 µs, from the first bit to the last.



007 · 帧是一串流动的比特 · A frame is a stream 0:56

按每米 5 纳秒算，8 微秒的信号有 1.6 公里长。我们整条总线才 4 米。

At 5 ns per metre, 8 µs of signal is 1.6 km long. Our whole line is 4 m.



008 · 办公室交换机的做法 · How an office switch does it 1:08

普通的办公室交换机，会先把整帧存下来，再往下传。

A typical office switch stores each frame before passing it on.

它等最后一个字节到齐，核对校验和，然后才开始发送。

It waits for the last byte, checks the checksum, then starts sending.

所以每过一站，至少要花整帧的时间：我们这一帧就是 8 微秒。

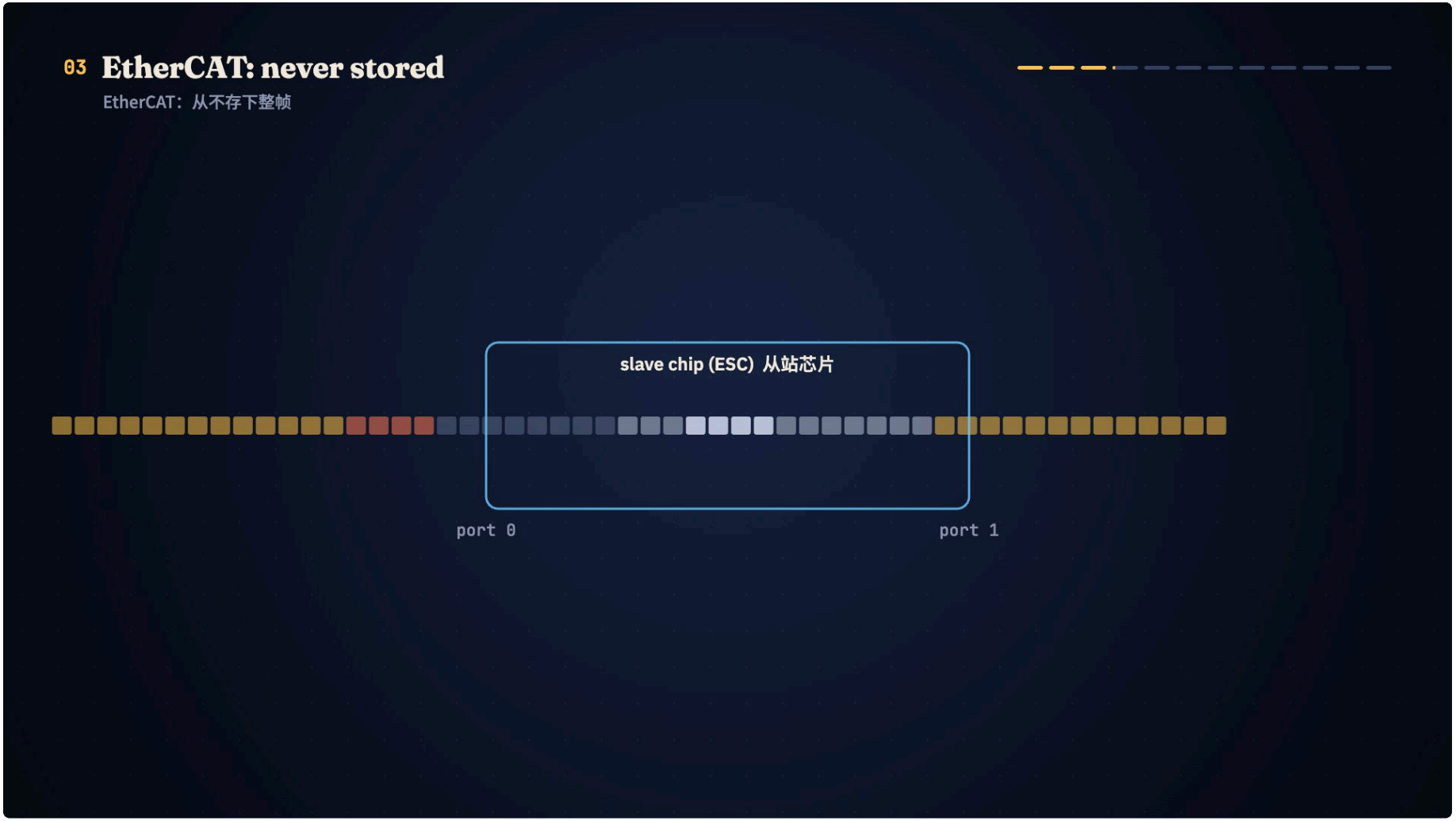
So every hop costs at least the whole frame time: 8 µs for our frame.



009 · 办公室交换机的做法 · How an office switch does it 1:28

就像一个邮局，每个包裹都要拆开验过，才寄往下一个城市。

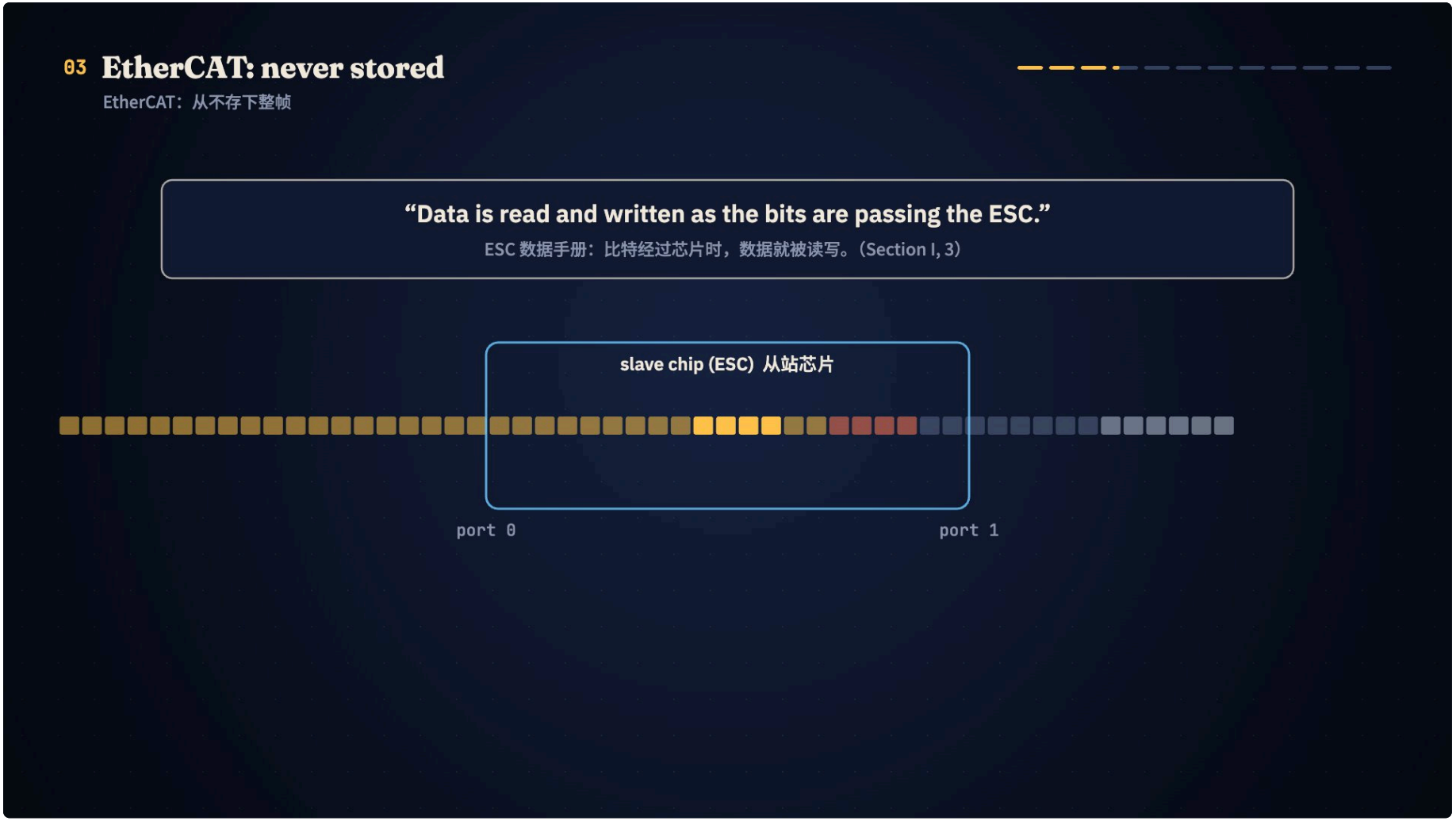
Like a post office that unpacks every parcel before sending it to the next town.



010 · EtherCAT：从不存下整帧 · EtherCAT: never stored 1:38

EtherCAT 的从站芯片不存帧。

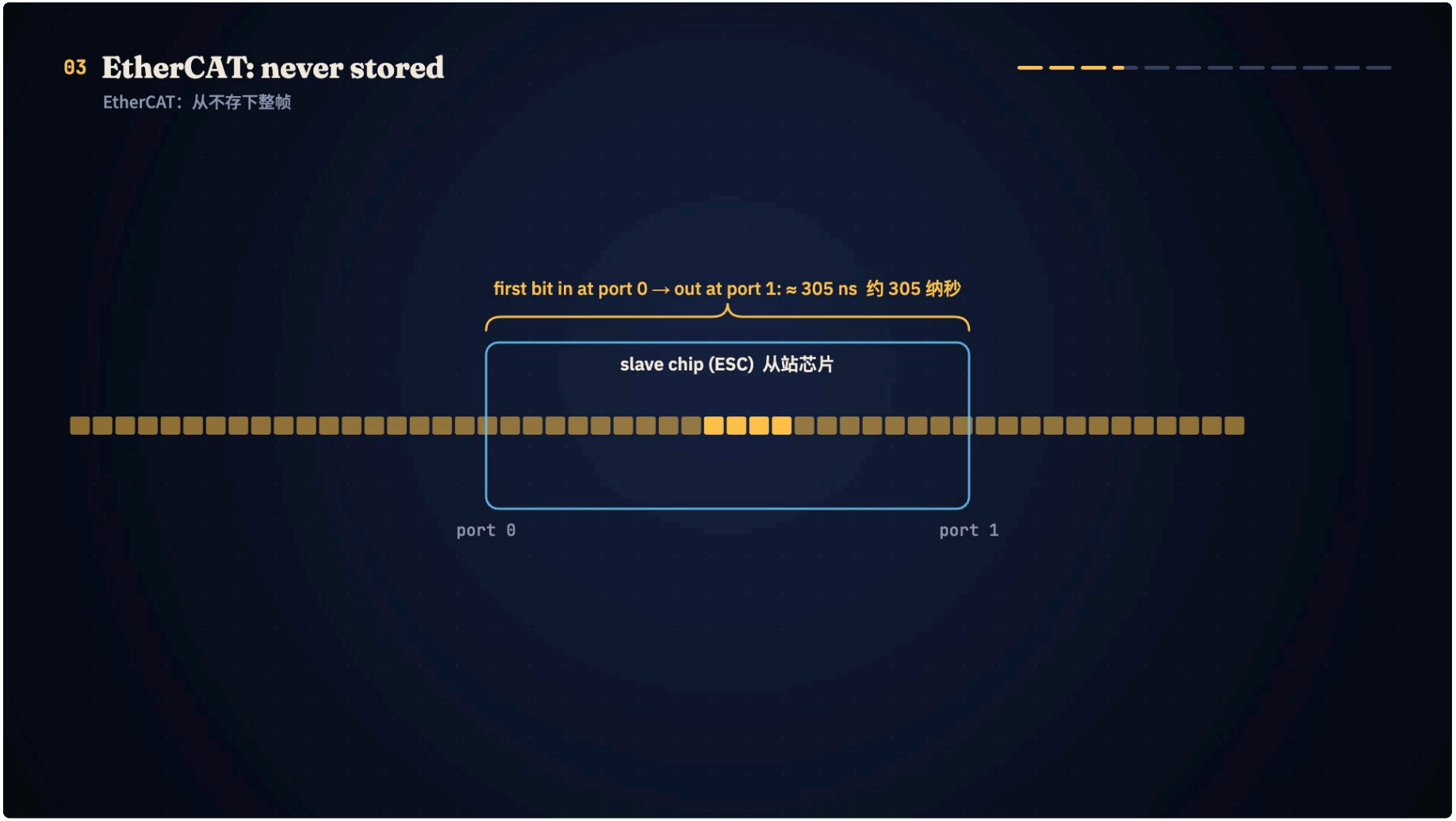
An EtherCAT slave chip does not store the frame.



011 · EtherCAT：从不存下整帧 · EtherCAT: never stored 1:43

它的数据手册说得很直白：比特经过时，数据就被读写。

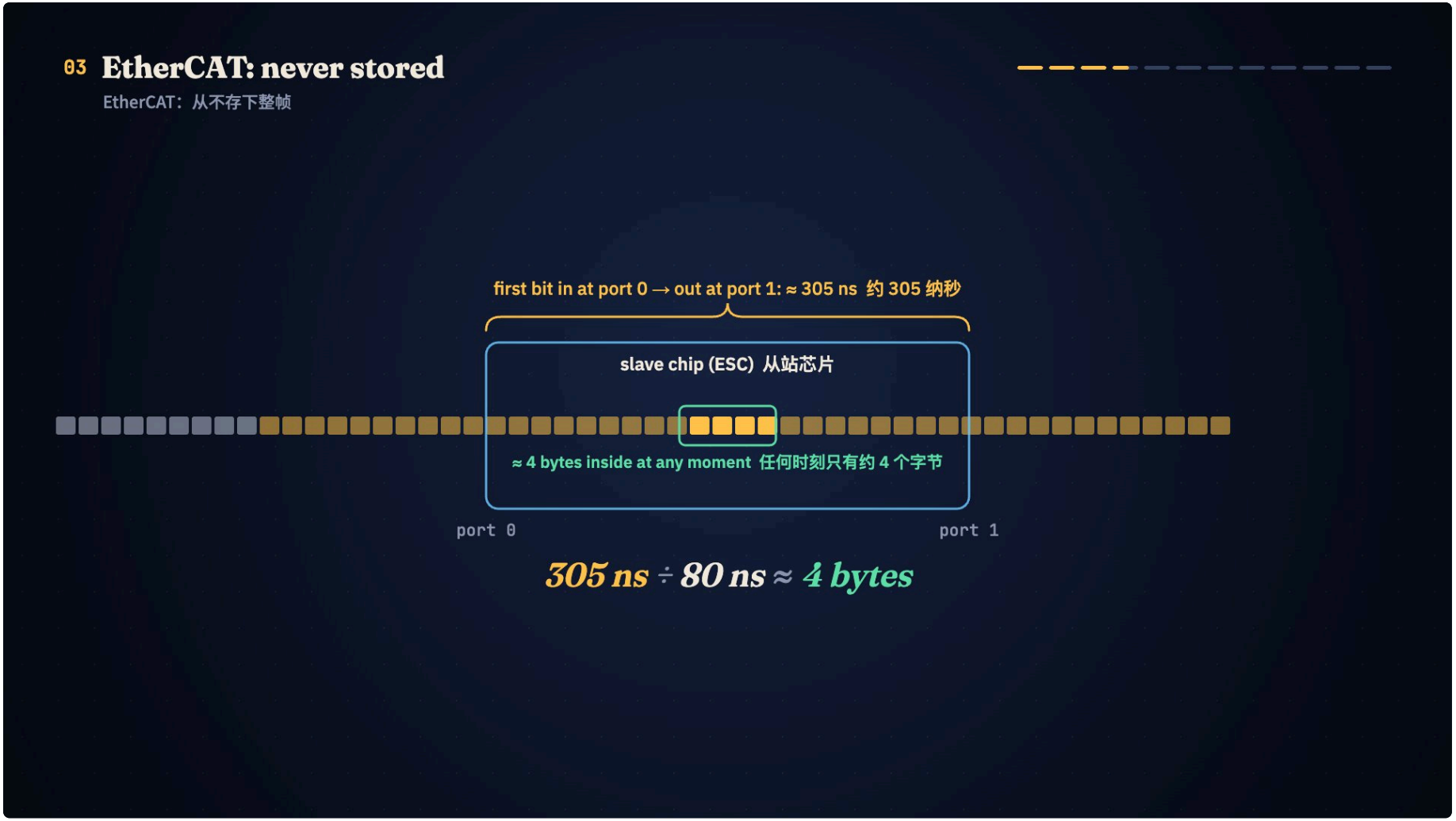
Its datasheet says it plainly: data is read and written as the bits pass.



012 · EtherCAT：从不存下整帧 · EtherCAT: never stored 1:50

比特从口 0 进来，大约 305 纳秒后就开始从口 1 出去。

The bits go in at port 0 and start leaving at port 1 about 305 ns later.



013 · EtherCAT：从不存下整帧 · EtherCAT: never stored 1:59

305 纳秒大约是 4 个字节，芯片任何时刻手里就只有这么多。

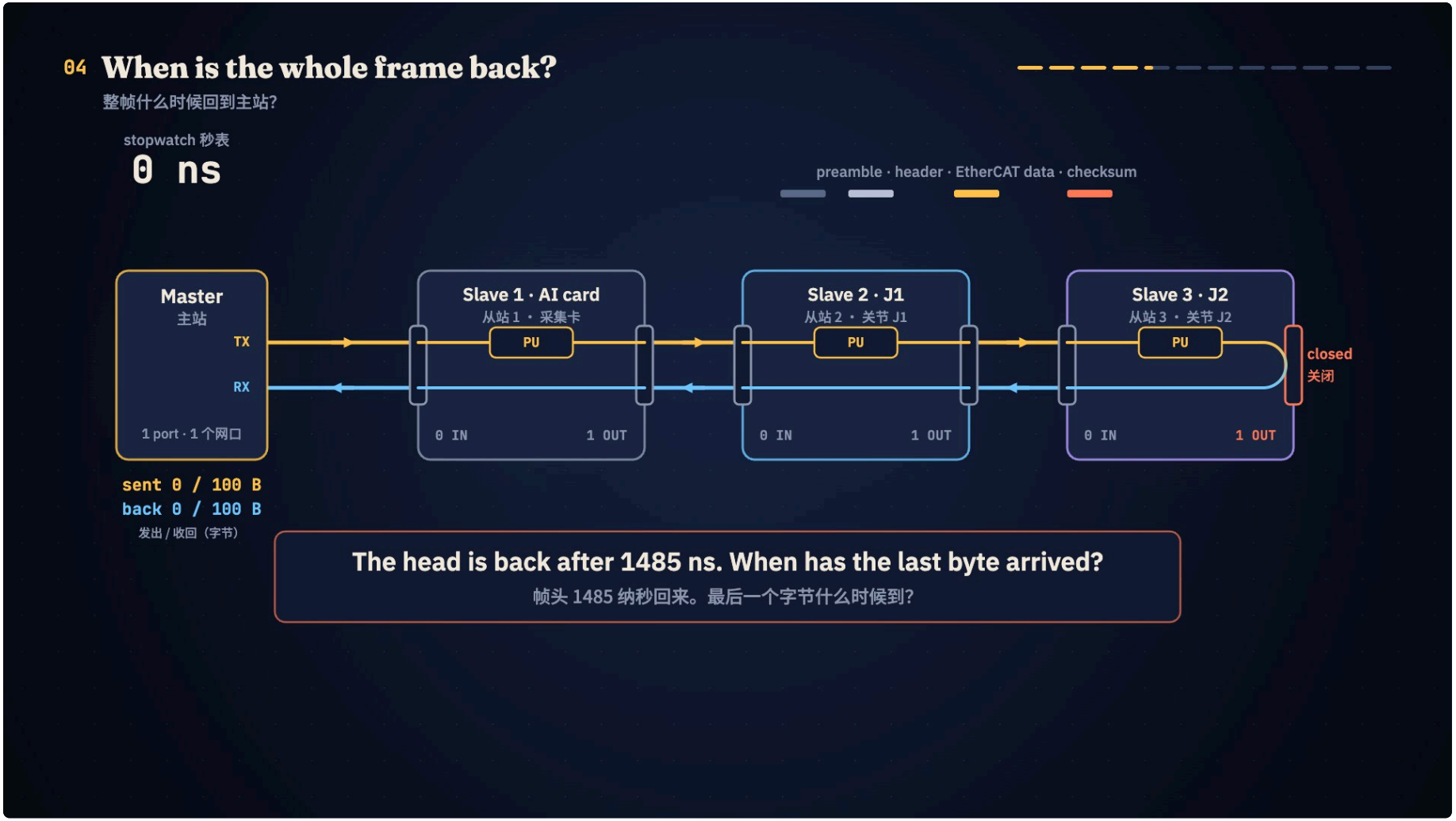
305 ns is about 4 bytes. That is all the chip holds at any moment.



014 · EtherCAT：从不存下整帧 · EtherCAT: never stored 2:06

就像一列不停站的火车：每到一站，站上的人趁车开过，取下寄给本站的信，再塞上本站要寄的信。

Like a train that never stops: at each station, a worker takes off the letters for this town and puts its own letters on, while the train rolls through.



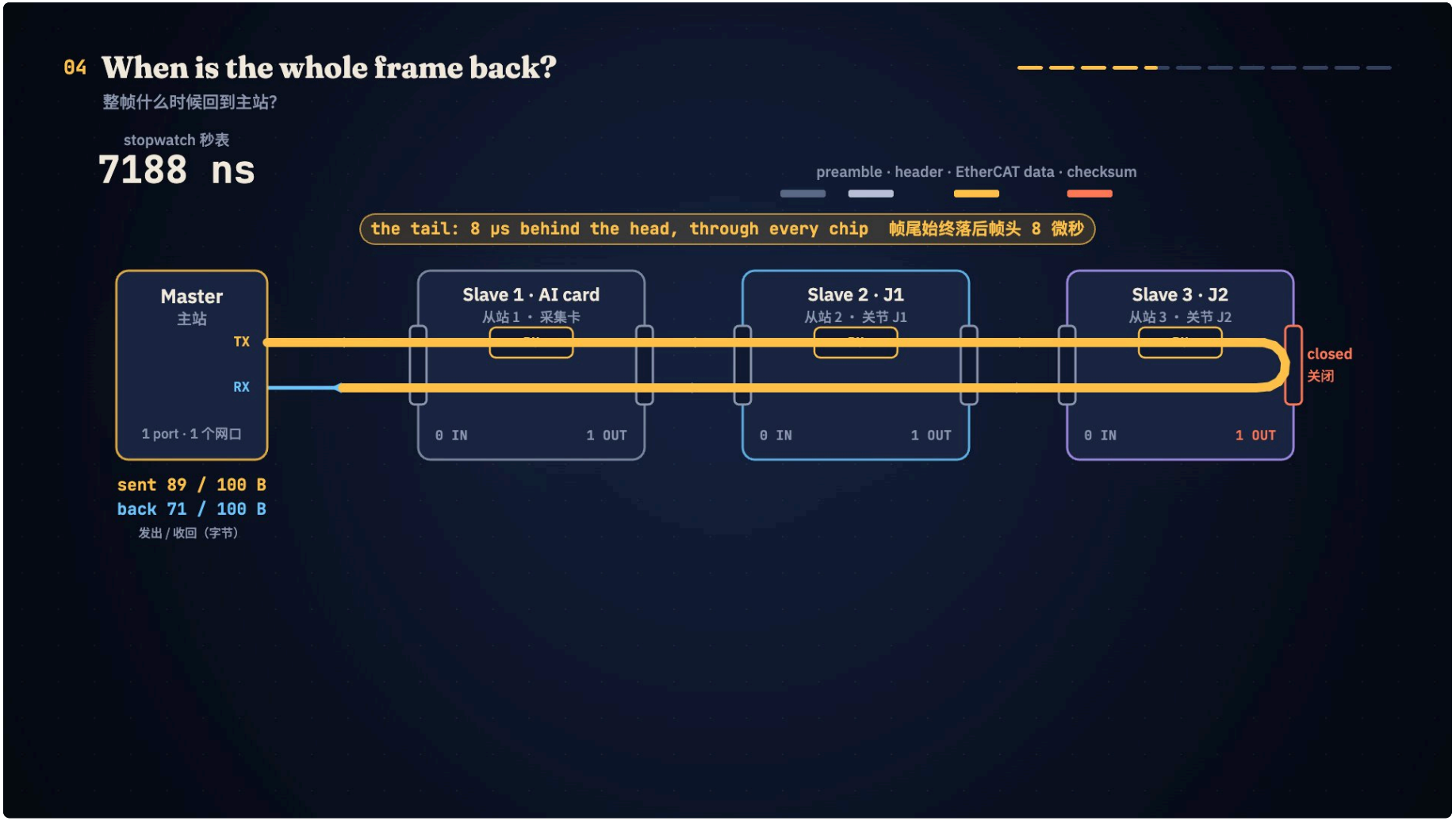
015 · 整帧什么时候回到主站? · When is the whole frame back? 2:22

那么，我们的 88 字节帧，在我们的总线上。

So, our 88-byte frame on our line.

帧头 1485 纳秒回来。最后一个字节什么时候到?

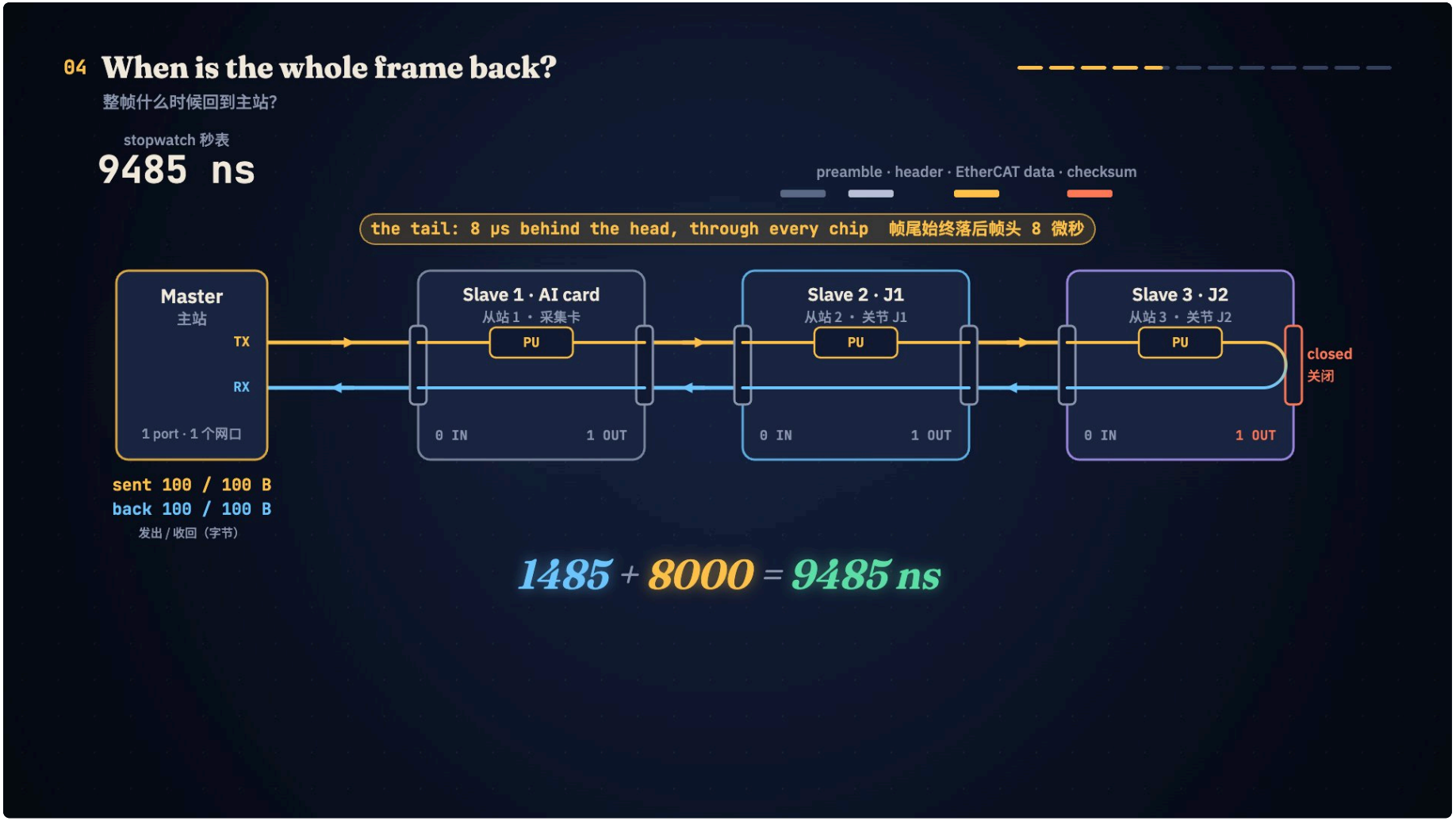
The head is back after 1485 ns. When has the last byte arrived?



016 · 整帧什么时候回到主站? · When is the whole frame back? 2:36

帧尾跟着帧头穿过每一颗芯片，始终落后 8 微秒。

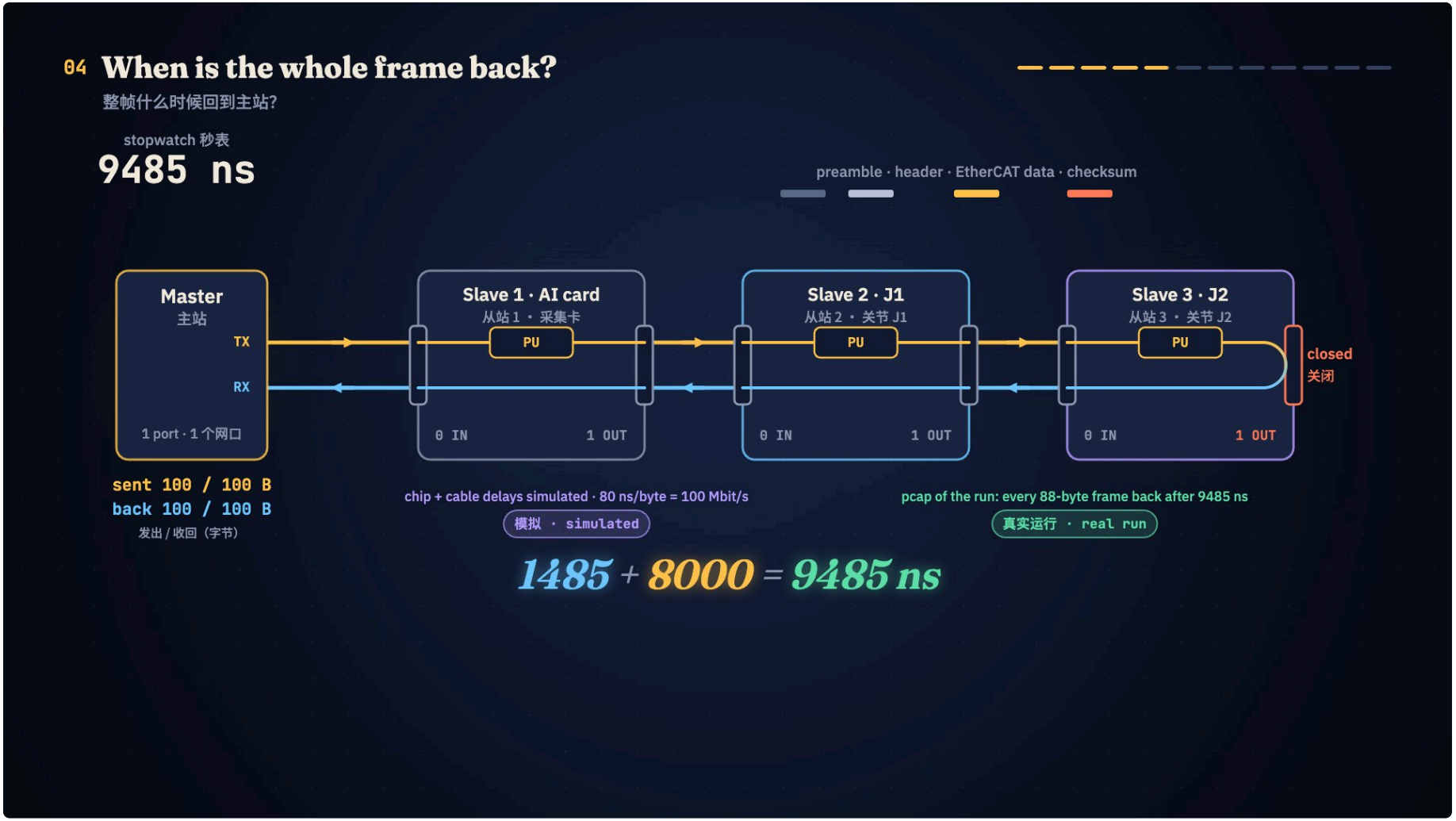
The tail follows the head through every chip, 8 µs behind it.



017 · 整帧什么时候回到主站? · When is the whole frame back? 2:42

1485 + 8000 = 9485 纳秒：整帧大约 9.5 微秒回到主站。

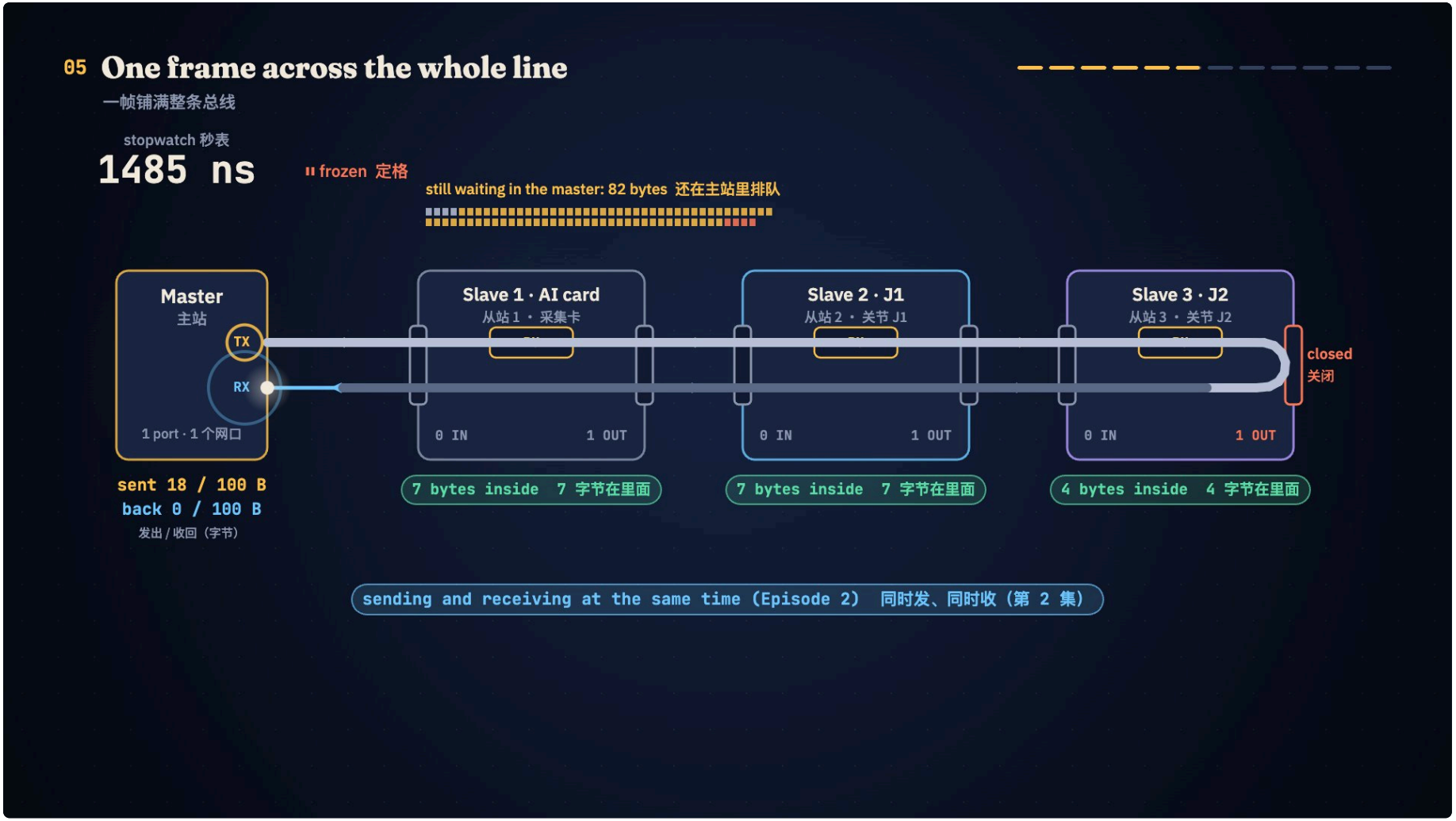
1485 + 8000 = 9485 ns. The whole frame is back after about 9.5 µs.



018 · 整帧什么时候回到主站? · When is the whole frame back? 2:50

芯片延迟是模拟的；每字节 80 纳秒是 100 Mbit/s 的标准。

The chip delays are simulated. The 80 ns per byte is the 100 Mbit/s standard.



019 · 一帧铺满整条总线 · One frame across the whole line 3:00

把时间定格在 1485 纳秒，帧头回家的那一刻。

Freeze time at 1485 ns, the moment the head comes home.

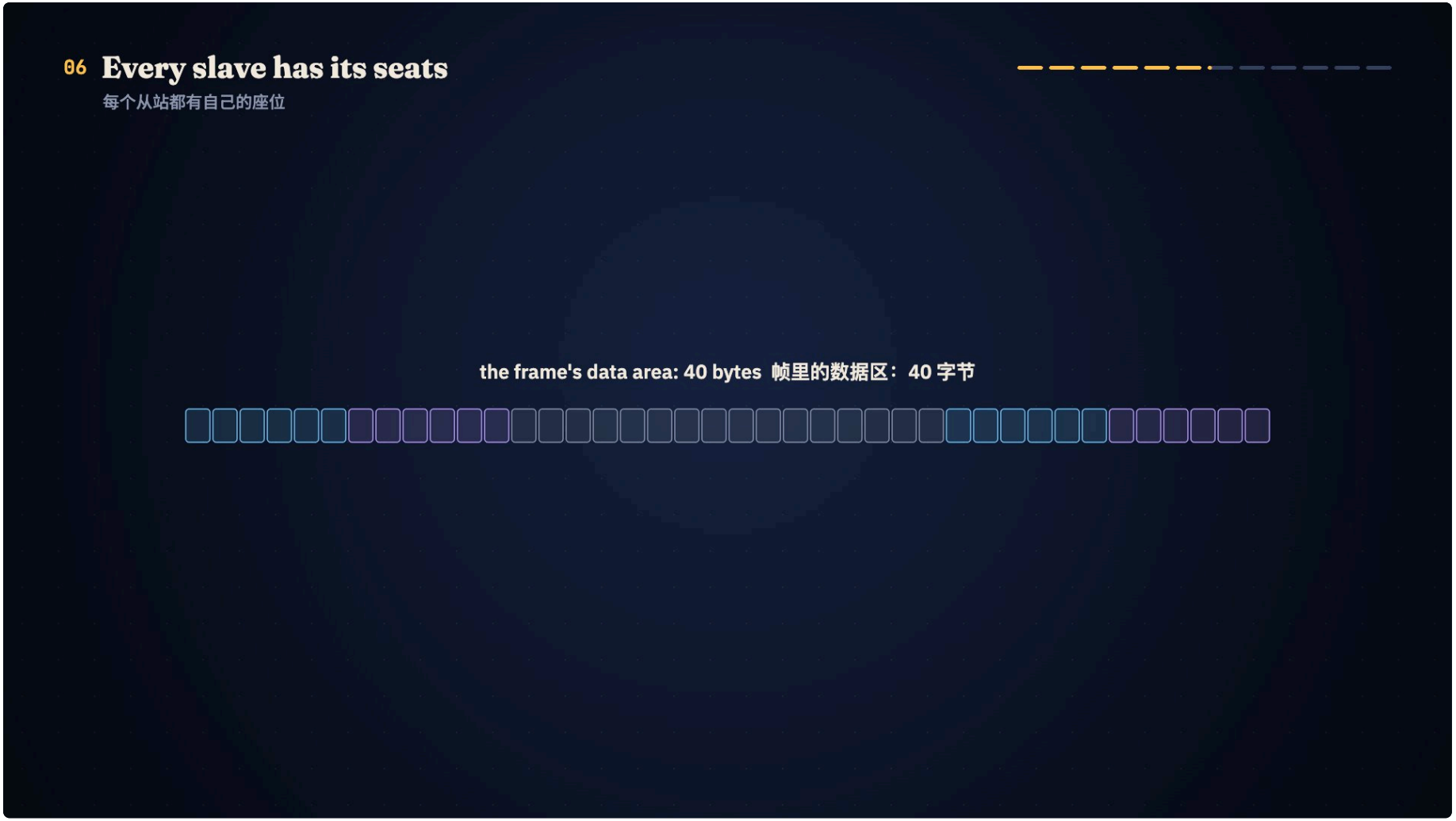
主站这时才发出去大约 18 个字节，其余的还在排队等着出门。

The master has sent only about 18 bytes so far. The rest is still waiting to go out.

这 18 个字节铺满了整个环：每个从站里都正有几个字节。

Those 18 bytes are spread over the whole ring: every slave has a few bytes inside it.

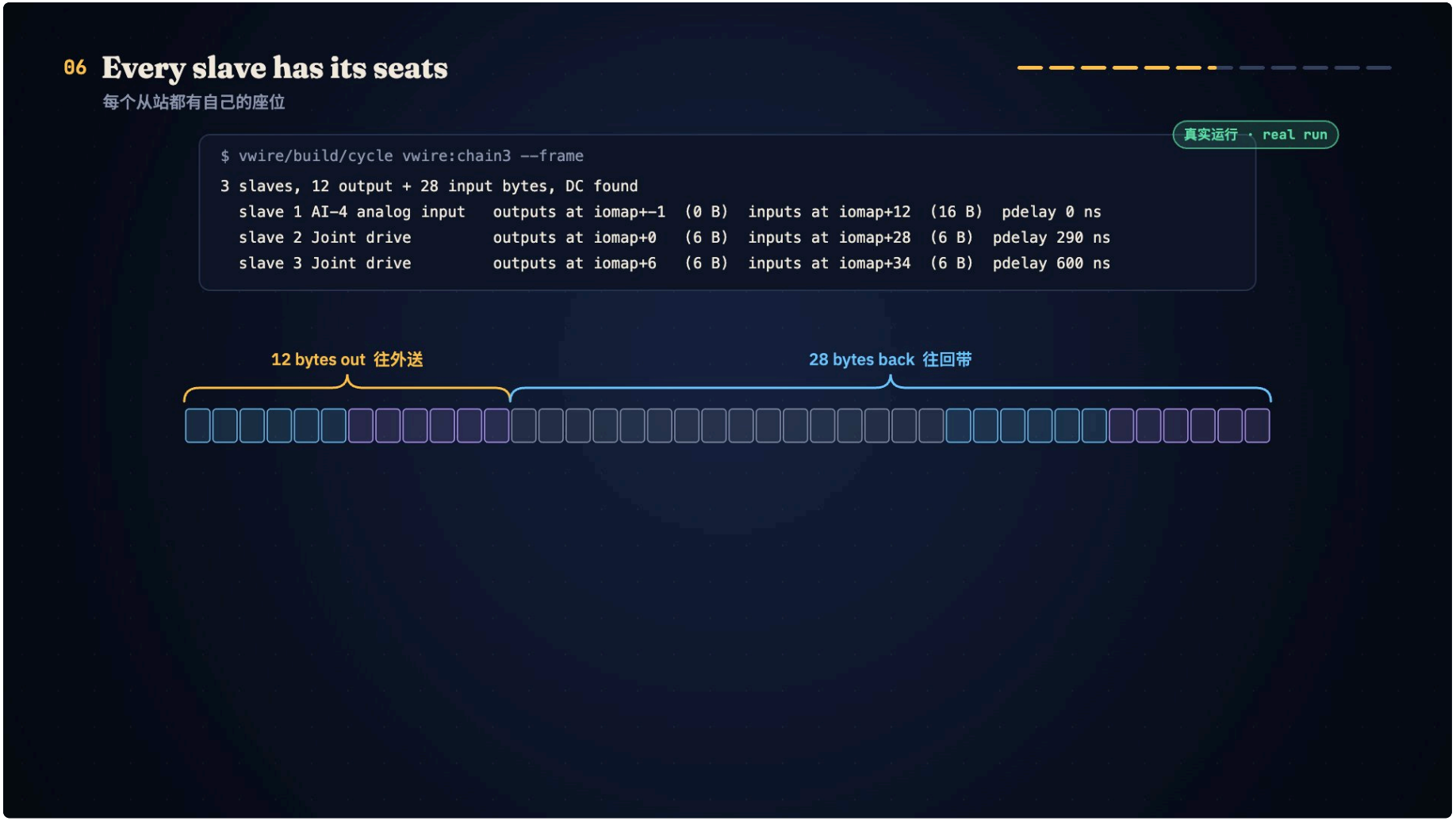
主站一对线发，一对线收，同时进行。这一切能成立，靠的就是这一点。



020 · 每个从站都有自己的座位 · Every slave has its seats 3:35

帧里的数据区，给每个从站都留了座位。

Inside the frame, the data area has a seat for each slave.



021 · 每个从站都有自己的座位 · Every slave has its seats 3:41

SOEM 打印出了它们的位置：12 字节往外送，28 字节往回带。

SOEM printed where they are: 12 bytes going out, 28 bytes coming back.



022 · 每个从站都有自己的座位 · Every slave has its seats 3:48

J1、J2 各有 6 字节的命令，再各有 6 字节写回答。

J1 and J2 each get 6 bytes of commands, and 6 bytes for their reply.

采集卡没有命令，只有 16 字节放它的读数。

The analog card has no commands, just 16 bytes for its readings.

座位怎么寻址，是第 3 集下篇的内容。这里只看它们怎么被填上。

How the seats are addressed is Episode 3b. Here, just watch them fill.



023 · 一帧真实的帧，一站一站看 · One real frame, slave by slave 4:11

这是运行中真实的一帧，每过一个从站就拍一张快照。

Here is one real frame from the run, copied after every slave.

离开主站时：带着 J1、J2 的新目标；回答座位里还是上个周期的值。

Leaving the master: new targets for J1 and J2. The reply seats still hold last cycle's values.

过了从站 1：采集卡把最新读数写进了自己的座位。

After slave 1: the card has written its newest readings into its seats.

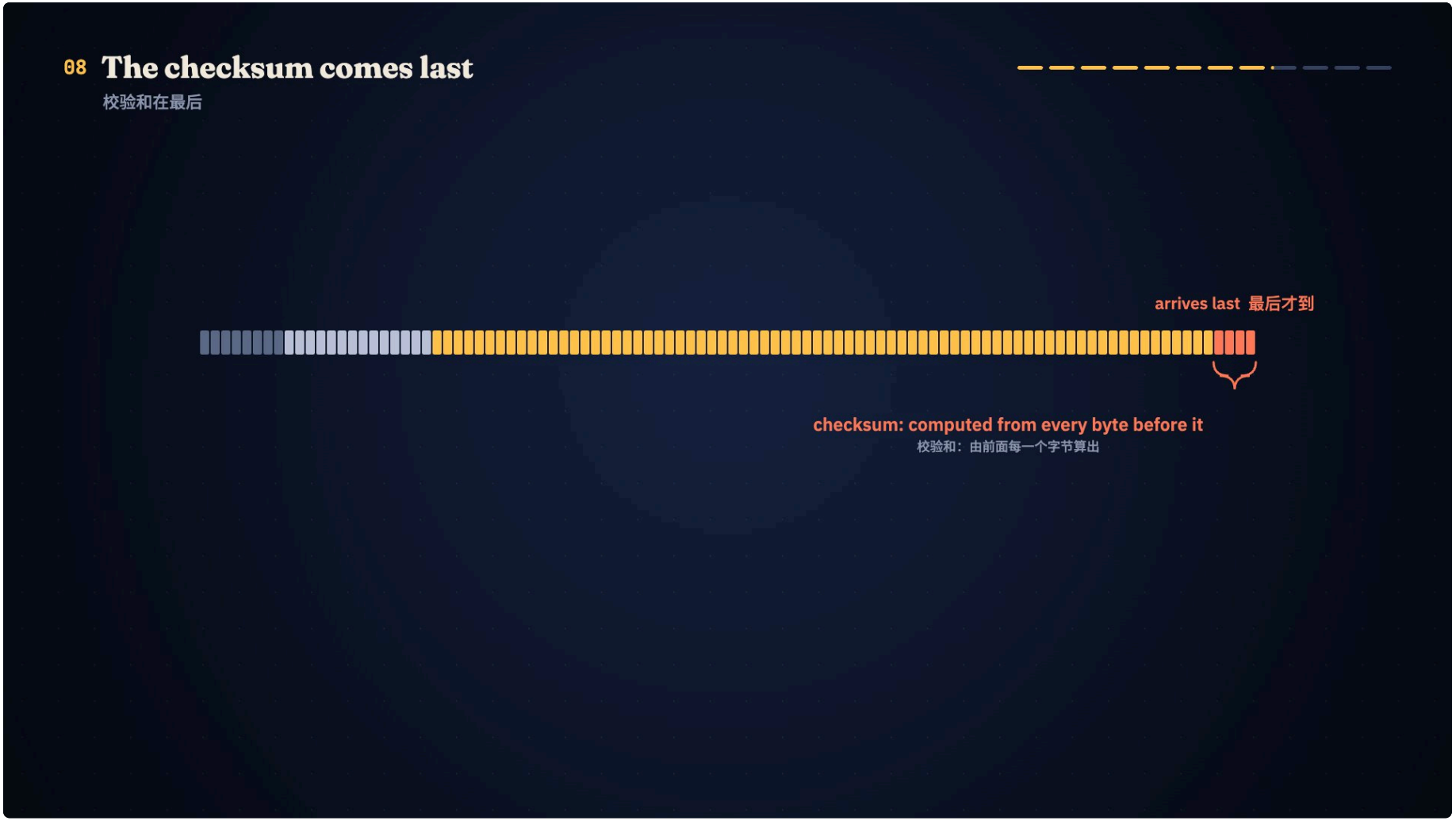
过了 J1：从取走了目标，把实际位置写进了自己的座位



024 · 一帧真实的帧，一站一站看 · One real frame, slave by slave 4:54

没有谁在等谁。每个从站都是在帧经过的那一刻干完自己的事。

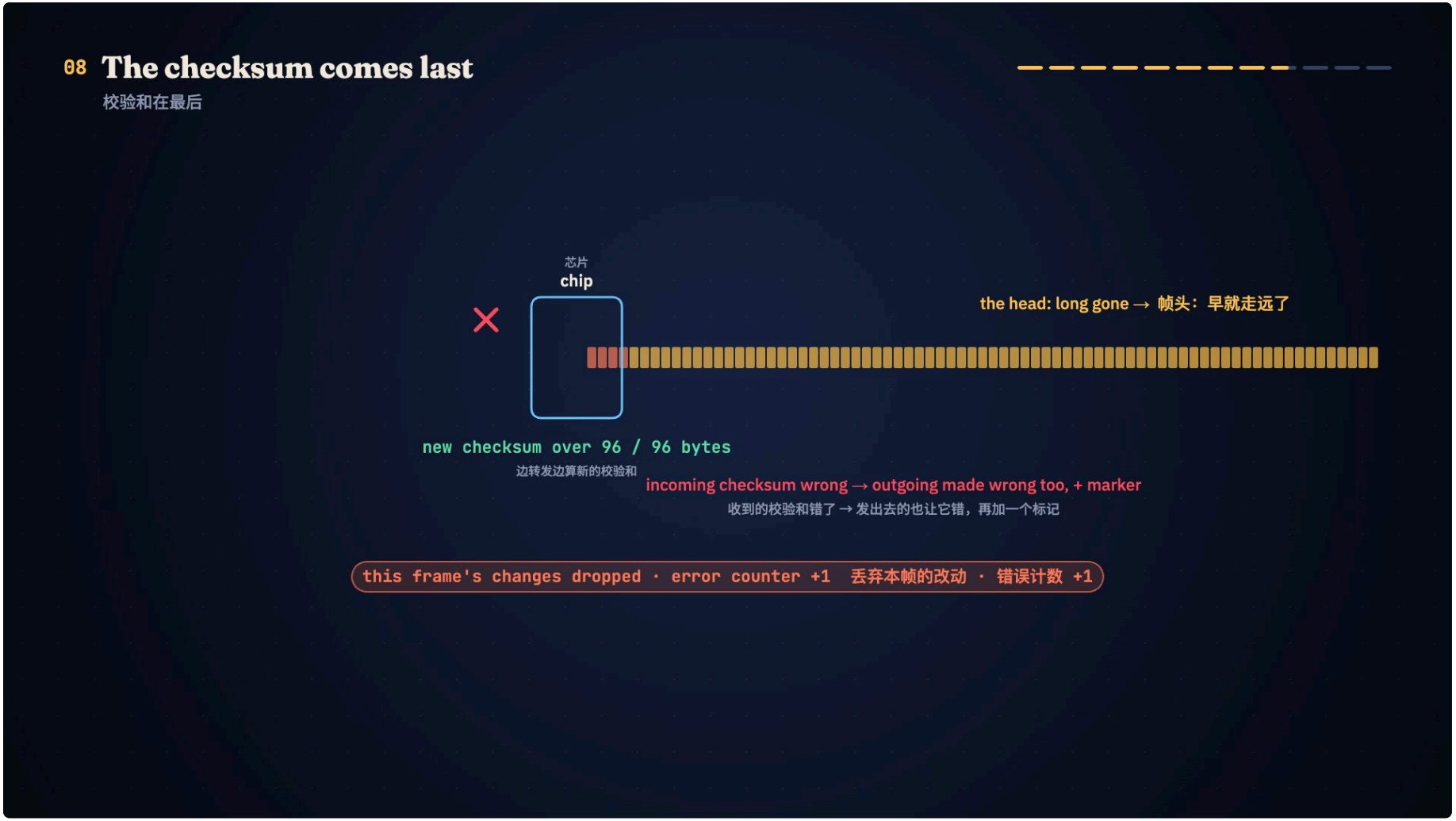
Nothing waited for anything. Each slave did its part while the frame was passing.



025 · 校验和在最后 · The checksum comes last 5:03

有个难题：每个以太网帧末尾都有校验和，用来证明字节完好无损。

One catch: every Ethernet frame ends with a checksum, proof that the bytes arrived intact.



026 · 校验和在最后 · The checksum comes last 5:11

交换机先核对它，再往外发。芯片做不到：校验和到的时候，帧的前面早就走远了。

A switch checks it before sending anything. The chip can't: by then, the front of the frame left long ago.

所以芯片边转发边算一个新的校验和，放在帧的末尾。

So the chip works out a new checksum as the bytes pass, and puts it at the end.

如果收到的校验和是错的，它就让发出去的也错，再加一个标记。

If the incoming checksum was wrong, it makes the outgoing one wrong too, and adds a marker.

它丢弃这一帧本来要在它身上生效的改动，并记一次错误。



027 · 校验和在最后 · The checksum comes last 5:46

主站的网卡把坏帧扔掉。这个周期没有回答，下一个周期重新开始。

The master's network card throws the bad frame away. That cycle has no answer; the next one starts fresh.

这部分来自 ESC 数据手册。我们模拟的总线不会出现比特错误。

This comes from the ESC datasheet. Our simulated line has no bit errors.



028 · 为什么这很重要 · Why it matters 6:04

每个从站只添大约 0.3 微秒，跟帧多长无关。

Each slave adds about 0.3 µs, whatever the frame's length.

如果要存下整帧，每个从站要 8 微秒：大约多 26 倍。

Storing our frame would cost 8 µs per slave: about 26 times more.

一个最大的以太网帧，存下来要约 122 微秒；边走边处理，还是约 0.3 微秒。

A full-size Ethernet frame takes about 122 µs to store. On the fly, still about 0.3 µs.

所以，一帧能在每个周期服务一长串从站，时间还绰绰有余。



029 · 本集要点 · Recap 6:35

一、帧是一串流：我们这一帧在线上长 8 微秒，比整条总线长得多。

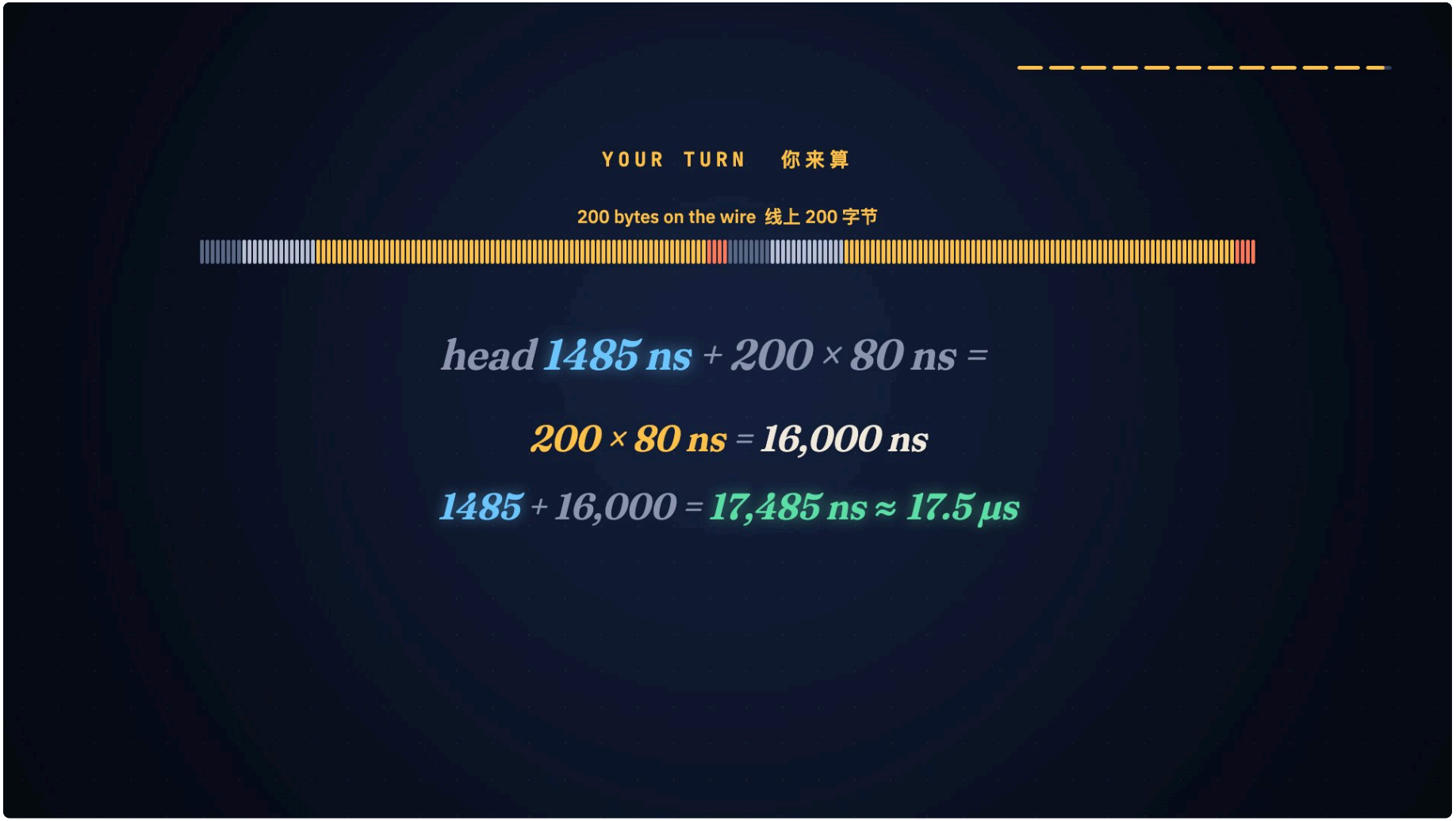
One: a frame is a stream. Ours is 8 µs long on the wire, far longer than the line.

二、每个从站在比特经过时读写自己的座位，手里只留几个字节。

Two: each slave reads and writes its seats as the bits pass, holding only a few bytes.

三、校验和没法先核对，所以坏帧在末尾被标记，最后被丢弃。

Three: the checksum can't be checked first, so a bad frame is marked and dropped at the end.



030 · 你来算 · Your turn 7:04

还是这条总线，这次帧在线上有 200 字节。

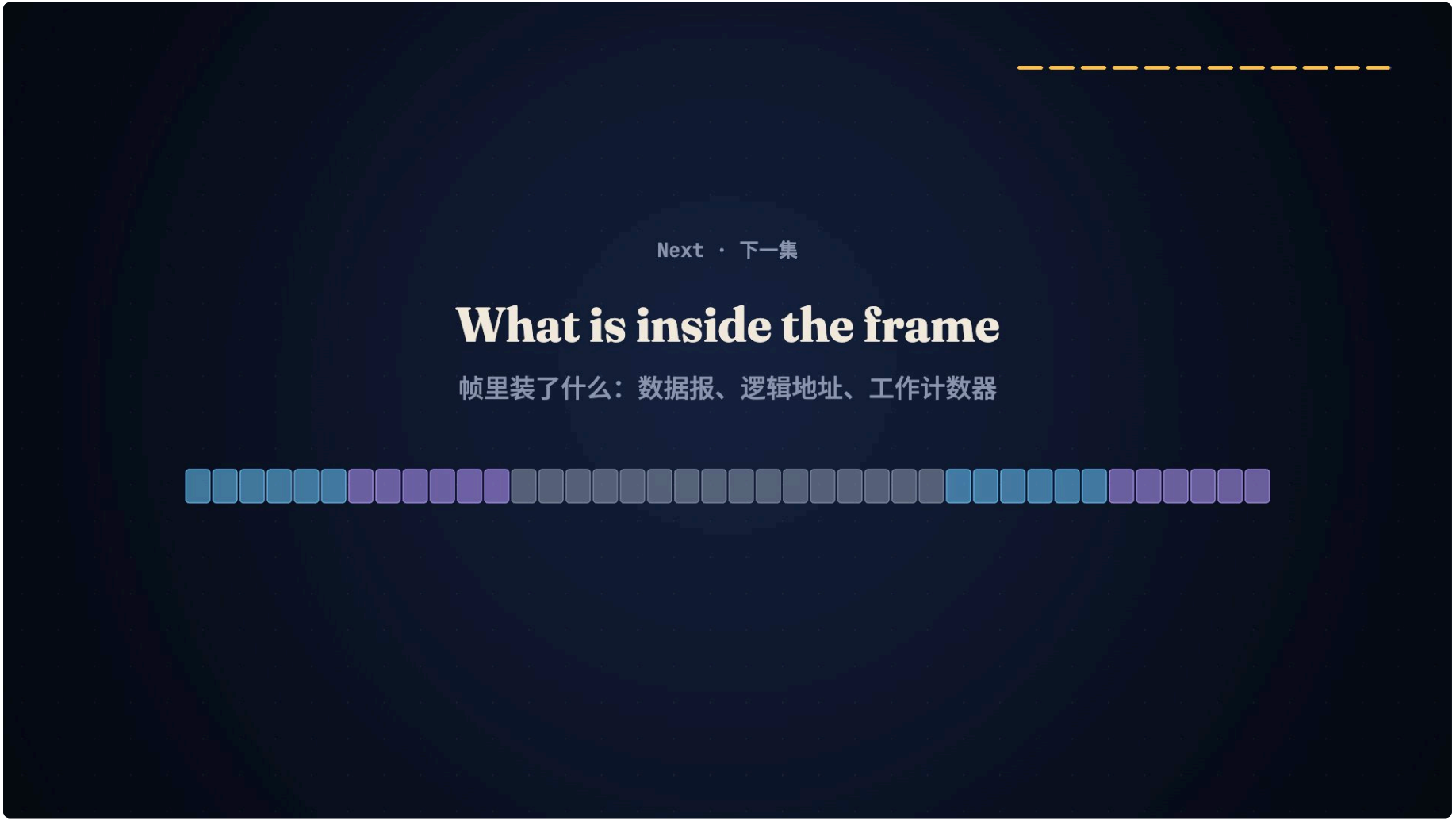
Same line. This time the frame is 200 bytes on the wire.

帧头仍然 1485 纳秒回来。整帧什么时候到齐？

The head is still back after 1485 ns. When has the whole frame arrived?

200 × 80 纳秒 = 16000 纳秒。1485 + 16000 = 17485 纳秒，约 17.5 微秒。

200 × 80 ns = 16,000 ns. 1485 + 16,000 = 17,485 ns, about 17.5 µs.



031 · 你来算 · Your turn 7:28

下一集：帧里装了什么。数据报、逻辑地址，还有工作计数器。

Next: what is inside the frame. Datagrams, logical addresses, and the working counter.

练习 · Practice

先自己答，再看答案。每个答案都写了依据：源码行，或者真实运行。 · Answer first, then open the answer. Each one says what it rests on: source lines or a real run.

第 1 题 · Q1 视频里的题 · From the video 2:21

我们的 88 字节周期帧，帧头 1485 纳秒回到主站。最后一个字节什么时候到？

Our 88-byte cyclic frame's head is back after 1485 ns. When has its last byte arrived?

9485 纳秒，约 9.5 微秒。线上一共 100 字节（8 前导码 + 88 + 4 校验和），每字节 80 纳秒，帧尾比帧头晚 8000 纳秒。

After 9485 ns, about 9.5 µs. 100 bytes go on the wire (8 preamble + 88 + 4 checksum) at 80 ns each, so the tail is 8000 ns behind the head.

依据 · Evidence

运行的 pcap 记录了每一帧发出和整帧收齐的时刻：每个 88 字节帧都是 9485 纳秒（data.py 逐帧检查）。芯片和网线延迟是模拟的，每字节 80 纳秒是 100 Mbit/s 的标准。 / The run's pcap records when each frame left and when it was completely back: 9485 ns for every 88-byte frame (data.py checks each one). Chip and cable delays are simulated; 80 ns per byte is 100 Mbit/s.

`episodes/ep03a-on-the-fly/data/runs.sh`

第 2 题 · Q2 视频里的题 · From the video 按代码推演 · Worked out from the code 2:59

帧头回到主站的那一刻（1485 纳秒），主站已经发出去多少字节？它们在哪里？

At the moment the head comes home (1485 ns), how many bytes has the master sent, and where are they?

约 18 字节（1485 ÷ 80 ≈ 18.6）。它们铺满了整个环：每个从站里都正有几个字节，其余 82 字节还在主站里排队。

About 18 (1485 ÷ 80 ≈ 18.6). They are spread around the whole ring, a few bytes inside every slave; the other 82 are still waiting in the master.

依据 · Evidence

第 k 个字节在 k × 80 纳秒时出门，此后沿着和帧头一样的路径走。 / Byte k leaves at k × 80 ns and then follows the same path as the head.

第 3 题 · Q3 视频里的题 · From the video 4:11

一帧真实的周期帧经过 J1 之后，数据区里哪些字节变了？

After a real cyclic frame passes J1, which bytes of its data area have changed?

只有 J1 自己的回答座位（数据区第 28–33 字节）：它写入了实际位置。J1 的命令座位只是被读取，别人的座位没动。

Only J1's own reply seats (bytes 28–33 of the data area), where it wrote its actual position. Its command seats were only read, and nobody else's seats changed.

依据 · Evidence

运行里把同一帧在每个从站之后都复制了一份；data.py 检查每一站改动的字节都只落在它自己的座位里。 / The run copies the same frame after every slave; data.py checks that the bytes each slave changes all fall in its own seats.

`episodes/ep03a-on-the-fly/data/runs.sh`

第 4 题 · Q4 视频里的题 · From the video 按代码推演 · Worked out from the code 5:02

一个从站收到一帧，末尾的校验和是错的。它能把这一帧拦下来吗？它会怎么做？

A slave receives a frame whose checksum at the end is wrong. Can it stop the frame? What does it do?

拦不下来：校验和到的时候，帧的前面早就发出去了。它让发出去的校验和也变成错的、加一个标记，丢弃这一帧本来要在它身上生效的改动，并记一次错误。主站网卡最后把坏帧扔掉。

No: by the time the checksum arrives, the front of the frame has already gone on. It makes the outgoing checksum wrong too and adds a marker, drops the changes this frame would have made in it, and counts an error. The master's network card throws the frame away.

依据 · Evidence

ESC 数据手册第 I 部分第 14 节（错误计数器）。模拟总线没有比特错误，所以这一题没有运行数据。 / ESC datasheet Section I, chapter 14 (error counters). Our simulated line has no bit errors, so no run backs this one.

第 5 题 · Q5 视频里的题 · From the video 按代码推演 · Worked out from the code 7:03

还是这条总线，帧在线上有 200 字节。帧头 1485 纳秒回来，整帧什么时候到齐?

Same line, a frame of 200 bytes on the wire. The head is back after 1485 ns. When has the whole frame arrived?

1485 + 200 × 80 = 17485 纳秒，约 17.5 微秒。

1485 + 200 × 80 = 17,485 ns, about 17.5 µs.

依据 · Evidence

帧尾比帧头晚 字节数 × 80 纳秒。 / The tail is (bytes × 80 ns) behind the head.

第 6 题 · Q6 新题 · New 按代码推演 · Worked out from the code

如果每个从站都像交换机那样先存整帧再转发，我们这一帧每过一个从站要多花多少时间?

If every slave stored the whole frame before passing it on, like a switch, how much would each slave add for our frame?

至少 8 微秒（整帧的线上时间），而边走边处理只要约 0.3 微秒，大约多 26 倍。

At least 8 µs (the frame's whole time on the wire), against about 0.3 µs on the fly: roughly 26 times more.

依据 · Evidence

8000 ÷ 305 ≈ 26。存储转发必须等最后一个字节到齐。 / 8000 ÷ 305 ≈ 26. Store-and-forward has to wait for the last byte.